

CSC494 Project Proposal

Error Propagation in Numerical Integration for Residual Neural ODE Parameter Estimation

Lu Wang

1 Introduction

Ordinary differential equations (ODEs) arise in many scientific applications, where unknown model parameters must often be estimated from observed trajectory data. Classical approaches to ODE parameter estimation rely on numerical integration and sensitivity analysis to support gradient-based optimization.

Recent Neural ODE methods incorporate machine learning into this framework by using neural networks to represent unknown or partially modeled dynamics. In hybrid residual Neural ODE models, known physics-based dynamics are combined with learned correction terms, offering a flexible approach to model misspecification.

However, since Neural ODE training relies on repeated numerical integration, errors introduced by the numerical solver may not only influence trajectory approximations, but also affect sensitivity computations, gradient estimates, and ultimately learned model parameters. This interaction is not yet well understood.

This project will investigate how numerical integration errors propagate through the training pipeline in residual Neural ODE parameter estimation.

2 Research Objectives

The project aims to study:

1. How numerical integration error propagates into sensitivity and gradient calculations during Neural ODE training.
2. How gradient and sensitivity errors influence learned parameter estimates.
3. Whether improvements in trajectory accuracy necessarily correspond to improved parameter recovery.
4. How these effects vary across numerical integration methods and discretization step sizes.

3 Model Formulation

The main goal of the project will study residual Neural ODE models of the form

$$\frac{dx}{dt} = f_{phys}(x) + r_{\theta}(x)$$

where:

- $f_{phys}(x)$ represents known mechanistic dynamics,
- $r_{\theta}(x)$ is a neural network representing learned residual corrections.

This formulation combines classical model-based dynamics with data-driven components.

Note: We have to penalize $r_{\theta}(x)$, or the expression may be dominated by $r_{\theta}(x)$ and it will come back to normal Neural ODE instead of residual Neural ODE.

However, there is difficulties of studying Residual Neural ODE directly: It is difficult to distinguish whether the learned residual term is compensating for missing physical dynamics or absorbing numerical solver errors. The decomposition between mechanistic dynamics and learned residual dynamics may become non-identifiable, making it difficult to determine how much of the system behavior is explained by each component and requiring appropriate regularization or inductive bias.

We will use

$$\frac{dx}{dt} = f_{phys}(x)$$

and

$$\frac{dx}{dt} = r_{\theta}(x)$$

as a baseline. For each model, we will first implement these two models and compare the error of residual neural network with them.

4 Methodology

The project will compare several numerical integration schemes, including:

- Forward Euler
- Classical Runge-Kutta (RK4)
- Adaptive time-stepping methods
- Linear multistep methods (LLMs)

Experiments will vary:

- step size
- observational noise level
- dynamical system type

Benchmark systems may include:

- Lotka-Volterra predator-prey dynamics
- Damped nonlinear oscillator

Performance will be evaluated using:

- trajectory prediction error

$$E_{\text{traj}} = \frac{1}{N} \sum_i \|x_i^{\text{pred}} - x_i^{\text{ref}}\|^2$$

- parameter estimation error

Parameter estimation error will be evaluated using component-wise relative parameter error

$$E_i = \frac{|\hat{\theta}_i - \theta_i^*|}{|\theta_i^*|},$$

where $\hat{\theta}_i$ denotes the learned parameter and θ_i^* denotes the ground truth parameter.

- gradient error

$$E_{\text{grad}} = \|\nabla_{\theta} L_h - \nabla_{\theta} L_{\text{ref}}\|_2$$

- sensitivity to numerical discretization, evaluated by comparing trajectory prediction error, parameter estimation error, and gradient error across different numerical integration methods and discretization step sizes.

Tentative framework:

- JAX + Diffrax

5 Tentative Implementation

The project will be implemented using JAX and the Diffrax library for differentiable ODE solving. Residual Neural ODE models will be constructed using small multilayer perceptrons to represent residual dynamics.

Experiments will initially focus on low-dimensional benchmark systems, including damped nonlinear oscillators and Lotka–Volterra dynamics. Synthetic trajectory data will be generated using high-accuracy numerical solvers to provide reference solutions and ground truth parameters.

Classical parameter estimation methods and pure Neural ODE models will first be implemented as baseline models. Residual Neural ODE models will then be introduced and compared against these baselines. Comparisons with classical parameter estimation methods will help isolate the effects of residual correction terms on parameter recovery under numerical integration error. Comparisons with pure Neural ODE models will help distinguish effects arising from learned dynamics and solver-induced training behavior, independent of mechanistic parameter recovery.

6 Expected Contributions

This project aims to:

- characterize how numerical integration errors influence parameter estimation in residual Neural ODE models relative to classical parameter estimation methods,
- investigate how solver-induced gradient and sensitivity errors affect learned dynamics by comparing residual Neural ODEs with pure Neural ODE models,
- identify settings in which improved trajectory accuracy does not necessarily correspond to improved recovery of underlying physical parameters,
- provide practical insights into the tradeoff between numerical integration accuracy, computational cost, and learning quality in Neural ODE training.

7 Timeline

Weeks 1–3

literature review, iteration of the proposal, framework learning.

Weeks 4–6

Two baseline implementation Residual Neural ODE implementation.

Weeks 7–9

Numerical experiments and error propagation analysis.

Weeks 10–12

Final analysis and report preparation.

8 References

1. Calver, J. and Enright, W.
Numerical methods for computing sensitivities for ODEs and DDEs.
Numerical Algorithms, 74, 1101–1117 (2017).
2. Chen, R. T. Q. et al.
Neural Ordinary Differential Equations.
NeurIPS, 2018.